

HW5

```
def linear_function():
    """
    Implements a linear function:
        Initializes W to be a random tensor of shape (4,3)
        Initializes X to be a random tensor of shape (3,1)
        Initializes b to be a random tensor of shape (4,1)
    Returns:
        result -- runs the session for Y = WX + b
    """

    np.random.seed(1)

    ### START CODE HERE ### (4 lines of code)
    X = tf.constant(np.random.randn(3,1),name='X')
    W = tf.constant(np.random.randn(4,3),name='W')
    b = tf.constant(np.random.randn(4,1),name='b')
    Y = tf.matmul(W,X)+b
    ### END CODE HERE ###

    # Create the session using tf.Session() and run it with sess.run(Y)

    ### START CODE HERE ###
    sess = tf.Session()
    result = sess.run(Y)
    ### END CODE HERE ###

    # close the session
    sess.close()

    return result
```

結果：

```
result = [[-2.15657382]
 [ 2.95891446]
 [-1.08926781]
 [-0.84538042]]
```

tf.constant 中 name 是一個可自訂義的變數，

`X = tf.constant(np.random.randn(3,1),name='X')`，等號左邊的 X 是將此隨機矩陣存入 Python 的變數中，而 name 的 X 是將此隨機矩陣存入 tensorflow 的 X 中。

參考：<https://blog.csdn.net/xiaohuihui1994/article/details/81022043>

事實上本次作業很多用法已於 tensorflow2.0 以後移除。

```
def sigmoid(z):
    """
    Computes the sigmoid of z

    Arguments:
    z -- input value, scalar or vector

    Returns:
    results -- the sigmoid of z
    """

    ### START CODE HERE ### ( approx. 4 lines of code)
    # Create a placeholder for x. Name it 'x'.
    x = tf.placeholder(tf.float32,name='x')

    # compute sigmoid(x)
    sigmoid = tf.sigmoid(x)

    # Create a session, and run it. Please use the method 2 explained above.
    # You should use a feed_dict to pass z's value to x.
    with tf.Session() as sess:
        # Run session and call the output "result"
        result = sess.run(sigmoid,feed_dict={x:z}) #feed_dict將z的值傳給x

    ### END CODE HERE ###

    return result
```

結果：

```
sigmoid(0) = 0.5
sigmoid(12) = 0.9999938
```

```
def one_hot_matrix(labels, C):
    """
    Creates a matrix where the i-th row corresponds to the ith class number and
    corresponds to the jth training example. So if example j has label i,
    will be 1.

    Arguments:
    labels -- vector containing the labels
    C -- number of classes, the depth of the one hot dimension

    Returns:
    one_hot -- one hot matrix
    """

    ### START CODE HERE ###

    # Create a tf.constant equal to C (depth), name it 'C'. (approx. 1 line)
    C = tf.constant(C)

    # Use tf.one_hot, be careful with the axis (approx. 1 line)
    one_hot_matrix = tf.one_hot(labels,depth=C,axis=0)

    # Create the session (approx. 1 line)
    sess = tf.Session()

    # Run the session (approx. 1 line)
    one_hot = sess.run(one_hot_matrix)

    # Close the session (approx. 1 line). See method 1 above.
    sess.close()

    ### END CODE HERE ###

    return one_hot
```

結果：

```
one_hot = [[0. 0. 0. 1. 0. 0.]
            [1. 0. 0. 0. 0. 1.]
            [0. 1. 0. 0. 1. 0.]
            [0. 0. 1. 0. 0. 0.]]
```

```
def ones(shape):
    """
    Creates an array of ones of dimension shape

    Arguments:
    shape -- shape of the array you want to create

    Returns:
    ones -- array containing only ones
    """

    ### START CODE HERE ###

    # Create "ones" tensor using tf.ones(...). (approx. 1 line)
    ones = tf.ones(shape)

    # Create the session (approx. 1 line)
    sess = tf.Session()

    # Run the session to compute 'ones' (approx. 1 line)
    ones = sess.run(ones)

    # Close the session (approx. 1 line). See method 1 above.
    sess.close()

    ### END CODE HERE ###
    return ones
```

結果：

```
ones = [1. 1. 1.]
```

```
def create_placeholders(n_x, n_y):
    """
    Creates the placeholders for the tensorflow session.

    Arguments:
    n_x -- scalar, size of an image vector (num_px * num_px = 64 * 64 * 3 = 12288)
    n_y -- scalar, number of classes (from 0 to 5, so -> 6)

    Returns:
    X -- placeholder for the data input, of shape [n_x, None] and dtype "float"
    Y -- placeholder for the input labels, of shape [n_y, None] and dtype "float"

    Tips:
    - You will use None because it let's us be flexible on the number of examples
    In fact, the number of examples during test/train is different.
    """

    ### START CODE HERE ### (approx. 2 lines)
    X = tf.placeholder(tf.float32, shape=[n_x, None], name='Placeholder')
    Y = tf.placeholder(tf.float32, shape=[n_y, None], name='Placeholder_1')
    ### END CODE HERE ###

    return X, Y
```

結果：

```
X = Tensor("Placeholder_5:0", shape=(12288, ?), dtype=float32)
Y = Tensor("Placeholder_1_5:0", shape=(6, ?), dtype=float32)
```

```
def initialize_parameters():
    """
    Initializes parameters to build a neural network with tensorflow. The shapes are:
        W1 : [25, 12288]
        b1 : [25, 1]
        W2 : [12, 25]
        b2 : [12, 1]
        W3 : [6, 12]
        b3 : [6, 1]

    Returns:
    parameters -- a dictionary of tensors containing W1, b1, W2, b2, W3, b3
    """

    tf.set_random_seed(1) # so that your "random" numbers match ours

    ### START CODE HERE ### (approx. 6 lines of code)
    W1 = tf.get_variable("W1", [25, 12288], initializer=tf.contrib.layers.xavier_initializer(seed=1))
    b1 = tf.get_variable("b1", [25, 1], initializer=tf.zeros_initializer())
    W2 = tf.get_variable("W2", [12, 25], initializer=tf.contrib.layers.xavier_initializer(seed=1))
    b2 = tf.get_variable("b2", [12, 1], initializer=tf.zeros_initializer())
    W3 = tf.get_variable("W3", [6, 12], initializer=tf.contrib.layers.xavier_initializer(seed=1))
    b3 = tf.get_variable("b3", [6, 1], initializer=tf.zeros_initializer())
    ### END CODE HERE ###

    parameters = {"W1": W1,
                  "b1": b1,
                  "W2": W2,
                  "b2": b2,
                  "W3": W3,
                  "b3": b3}

    return parameters
```

結果：

```
W1 = <tf.Variable 'W1:0' shape=(25, 12288) dtype=float32_ref>
b1 = <tf.Variable 'b1:0' shape=(25, 1) dtype=float32_ref>
W2 = <tf.Variable 'W2:0' shape=(12, 25) dtype=float32_ref>
b2 = <tf.Variable 'b2:0' shape=(12, 1) dtype=float32_ref>
```

```
def forward_propagation(X, parameters):
    """
    Implements the forward propagation for the model: LINEAR -> RELU -> LINEAR -> RELU -> LINEAR -> SOFTMAX

    Arguments:
    X -- input dataset placeholder, of shape (input size, number of examples)
    parameters -- python dictionary containing your parameters "W1", "b1", "W2", "b2", "W3", "b3"
                  the shapes are given in initialize_parameters

    Returns:
    Z3 -- the output of the last LINEAR unit
    """

    # Retrieve the parameters from the dictionary "parameters"
    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['b2']
    W3 = parameters['W3']
    b3 = parameters['b3']

    ### START CODE HERE ### (approx. 5 Lines)
    Z1 = tf.matmul(W1, X) + b1
    A1 = tf.nn.relu(Z1)
    Z2 = tf.matmul(W2, A1) + b2
    A2 = tf.nn.relu(Z2)
    Z3 = tf.matmul(W3, A2) + b3
    ### END CODE HERE ###

    # Numpy Equivalents:
    # Z1 = np.dot(W1, X) + b1
    # A1 = relu(Z1)
    # Z2 = np.dot(W2, a1) + b2
    # A2 = relu(Z2)
    # Z3 = np.dot(W3, Z2) + b3

    return Z3
```

結果：

```
Z3 = Tensor("add_2:0", shape=(6, ?), dtype=float32)
```

```
def compute_cost(Z3, Y):
    """
    Computes the cost

    Arguments:
    Z3 -- output of forward propagation (output of the last LINEAR unit), of shape (6, number of examples)
    Y -- "true" labels vector placeholder, same shape as Z3

    Returns:
    cost - Tensor of the cost function
    """

    # to fit the tensorflow requirement for tf.nn.softmax_cross_entropy_with_logits(...,...)
    logits = tf.transpose(Z3)
    labels = tf.transpose(Y)

    ### START CODE HERE ### (1 line of code)
    cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=logits, labels=labels))
    ### END CODE HERE ###

    return cost
```

結果：

```
cost = Tensor("Mean:0", shape=(), dtype=float32)
```

```

def model(X_train, Y_train, X_test, Y_test, learning_rate = 0.0001,
         num_epochs = 1500, minibatch_size = 32, print_cost = True):
    """
    Implements a three-layer tensorflow neural network: LINEAR->RELU->LINEAR->RELU->LINEAR->SOFTMAX.

    Arguments:
    X_train -- training set, of shape (input size = 12288, number of training examples = 1080)
    Y_train -- test set, of shape (output size = 6, number of training examples = 1080)
    X_test -- training set, of shape (input size = 12288, number of training examples = 120)
    Y_test -- test set, of shape (output size = 6, number of test examples = 120)
    learning_rate -- learning rate of the optimization
    num_epochs -- number of epochs of the optimization loop
    minibatch_size -- size of a minibatch
    print_cost -- True to print the cost every 100 epochs

    Returns:
    parameters -- parameters learnt by the model. They can then be used to predict.
    """

    ops.reset_default_graph()
    tf.set_random_seed(1)
    seed = 3
    (n_x, m) = X_train.shape
    n_y = Y_train.shape[0]
    costs = []

    # Create Placeholders of shape (n_x, n_y)
    ### START CODE HERE ### (1 Line)
    X, Y = create_placeholders(n_x, n_y)
    ### END CODE HERE ###

    # Initialize parameters
    ### START CODE HERE ### (1 Line)
    parameters = initialize_parameters()
    ### END CODE HERE ###

    # Forward propagation: Build the forward propagation in the tensorflow graph
    ### START CODE HERE ### (1 Line)
    Z3 = forward_propagation(X, parameters)
    ### END CODE HERE ##

    ### START CODE HERE ### (1 Line)
    cost = compute_cost(Z3, Y)
    ### END CODE HERE ###

    # Backpropagation: Define the tensorflow optimizer. Use an AdamOptimizer.
    ### START CODE HERE ### (1 Line)
    optimizer = tf.train.AdamOptimizer(learning_rate=learning_rate).minimize(cost)
    ### END CODE HERE ###

    # Initialize all the variables
    init = tf.global_variables_initializer()

    # Start the session to compute the tensorflow graph
    with tf.Session() as sess:

        # Run the initialization
        sess.run(init)

        # Do the training loop
        for epoch in range(num_epochs):

            epoch_cost = 0.
            num_minibatches = int(m / minibatch_size) # number of minibatches of size minibatch_size in the train set
            seed = seed + 1
            minibatches = random_mini_batches(X_train, Y_train, minibatch_size, seed)

            for minibatch in minibatches:

                # Select a minibatch
                (minibatch_X, minibatch_Y) = minibatch

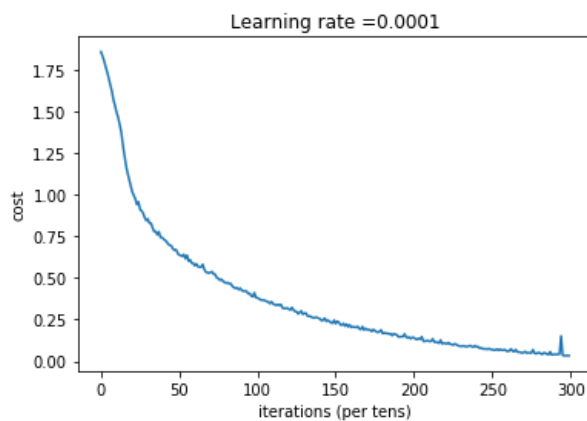
                # IMPORTANT: The line that runs the graph on a minibatch.
                # Run the session to execute the "optimizer" and the "cost", the feeddict should contain a minibatch for (X,Y).
                ### START CODE HERE ### (1 line)
                _, minibatch_cost = sess.run([optimizer, cost], feed_dict={X: minibatch_X, Y: minibatch_Y})
                ### END CODE HERE ###

            epoch_cost += minibatch_cost / num_minibatches

```

結果：

```
Cost after epoch 0: 1.855702
Cost after epoch 100: 1.016458
Cost after epoch 200: 0.733102
Cost after epoch 300: 0.572938
Cost after epoch 400: 0.468799
Cost after epoch 500: 0.380979
Cost after epoch 600: 0.313819
Cost after epoch 700: 0.254258
Cost after epoch 800: 0.203795
Cost after epoch 900: 0.166410
Cost after epoch 1000: 0.141497
Cost after epoch 1100: 0.107579
Cost after epoch 1200: 0.086229
Cost after epoch 1300: 0.059415
Cost after epoch 1400: 0.052237
```



討論

作業使用 Adam 當作優化器，預測準度 71%準確率，71%正確率並不算高，還是有很大的機率會誤判。

以下使用不同優化器當比較(*註)，為了提高準確度以及減少 **overfitting**，適當的調整各修化器參數以及 **learning rate**，學習次數則不改變，方便比較。

*註：優化器的使用參考

(1)

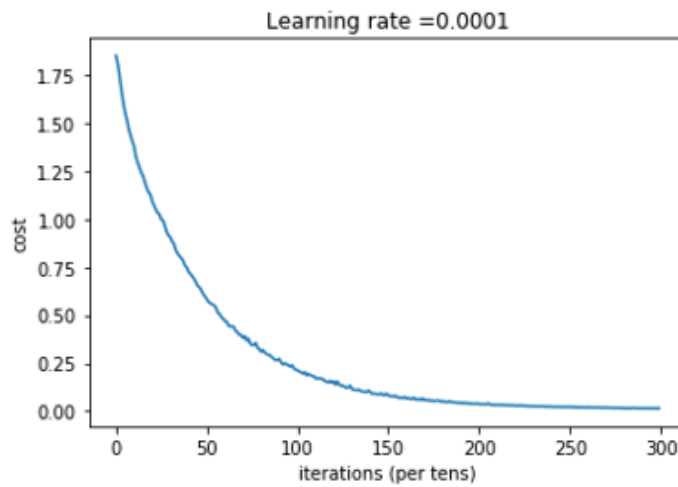
<https://medium.com/%E9%9B%9E%E9%9B%9E%E8%88%87%E5%85%94%E5%85%94%E7%9A%84%E5%B7%A5%E7%A8%8B%E4%B8%96%E7%95%8C/%E6%A9%9F%E5%99%A8%E5%AD%B8%E7%BF%92ml-note-sgd-momentum-adagrad-adam-optimizer-f20568c968db>

(2)

<https://zhuanlan.zhihu.com/p/34169434>

(一) MomentumOptimize

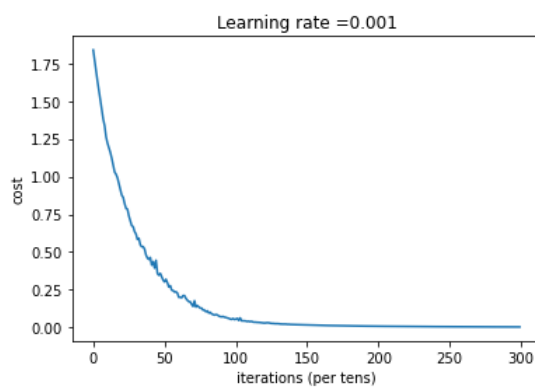
MomentumOptimize, momentum=0.9, learning rate=0.0001



```
Parameters have been trained!  
Train Accuracy: 1.0  
Test Accuracy: 0.8583335
```

(圖一)

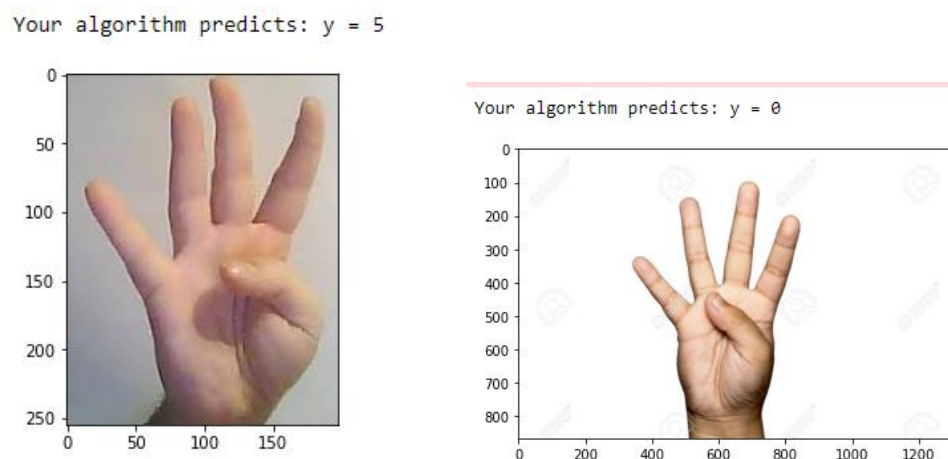
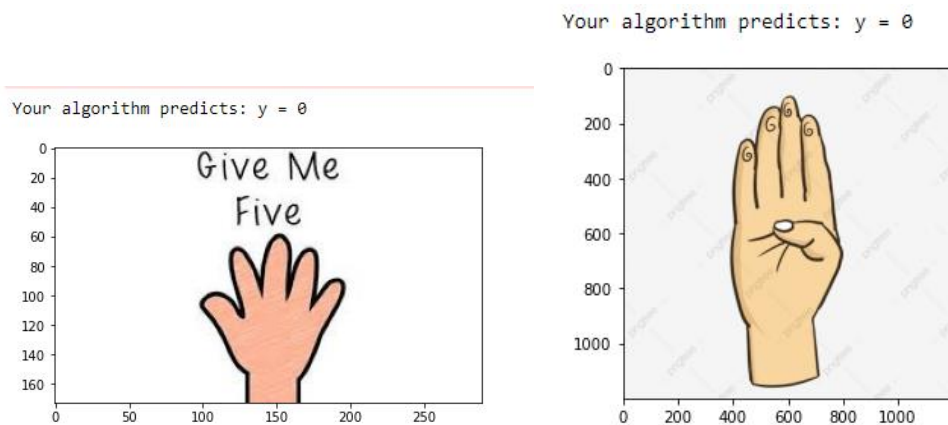
使用 momentum 當作優化器，雖然從訓練資料的準確度看起來有 overfitting 的問題，但預測準確度高達 85.8% 準確率。試著改變 momentum=0.5, learning rate=0.001, 準確率可提高至 87.5%，並在 google 隨便抓了一張比“YA”的圖判斷能正確預測為 2，繼續測試其他圖片，發現大部分判斷錯誤，overfitting 了。



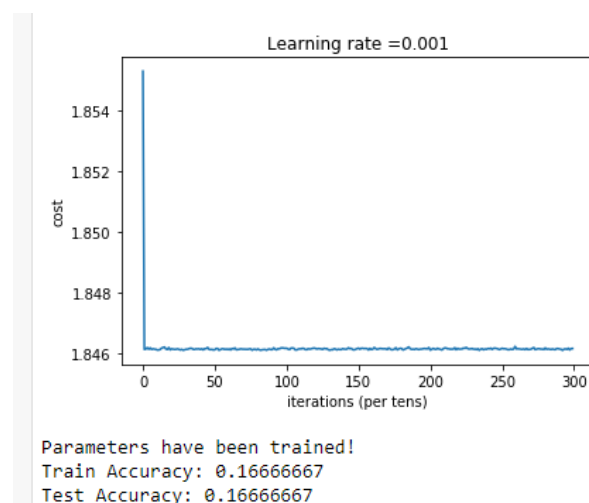
```
Parameters have been trained!  
Train Accuracy: 1.0  
Test Accuracy: 0.875
```

Your algorithm predicts: y = 2

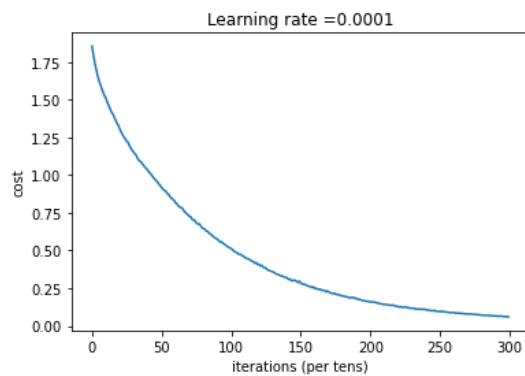




由於想讓正確率突破九成，修改了 $\text{momentum}=0.9$ ， $\text{learning rate}=0.001$ ，出來的結果是一個爛掉的模型，完全無法使用，我們可以從(圖一)發現，經過 200 次學習後就已開始收斂，將 learning rate 加大成 10 倍是非常不明智的選擇。



momentum=0.8，learning rate=0.0001

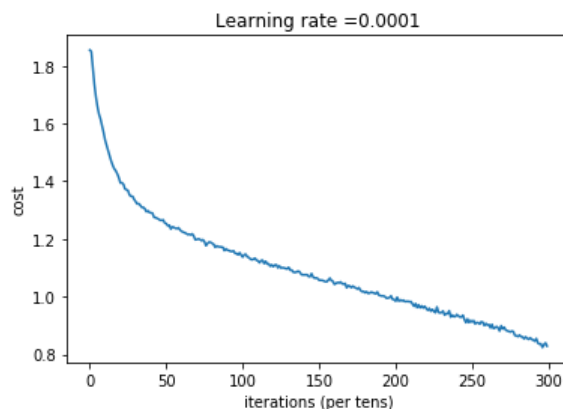


```
Parameters have been trained!  
Train Accuracy: 0.9962963  
Test Accuracy: 0.8666667
```

將 learning rate 改回 0.0001 就能正常使用，因此可以得知，learning rate 的選擇對於此優化器來說非常敏感，必須慎選。

(二) RMSPropOptimizer

使用 RMSPropOptimizer，decay=0.9, momentum=0

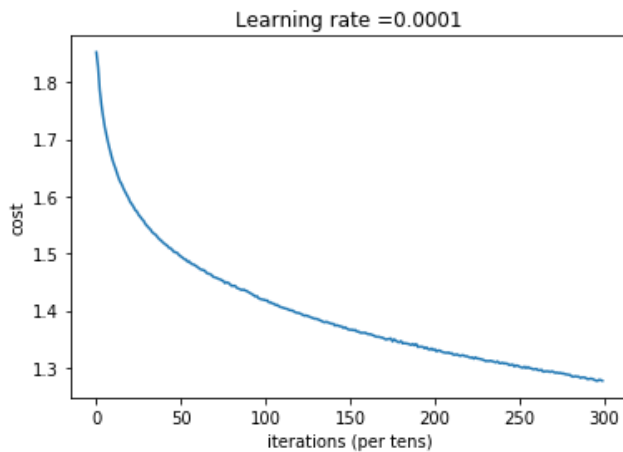


```
Parameters have been trained!  
Train Accuracy: 0.6722222  
Test Accuracy: 0.55
```

正確率非常低，但很明顯看到 cost 是可以繼續下降的，並未收斂，因此若把學習次數增加預期能提高準確率，這顯示此優化器對於此數據需要花較多的時間，若不要花太多時間可以嘗試更改 learning rate。

(三) AdagradOptimizer

使用 AdagradOptimizer，initial_accumulator_value=0.1

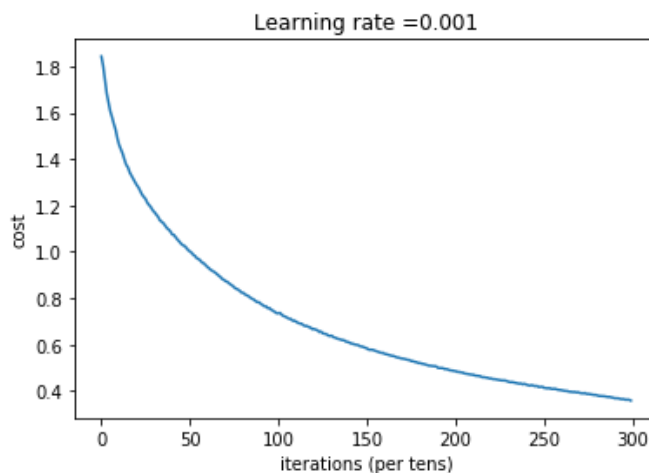


Parameters have been trained!
Train Accuracy: 0.5194447
Test Accuracy: 0.4916667

↩

正確率也是很低，和 RMSPropOptimizer 同等狀況，cost 尚未有收斂趨勢，所以在不改變學習次數下，可以合理修改 learning rate 使正確率上升。

將 AdagradOptimizer 的 learning rate 調整為 0.001，正確率可以來到 84%。

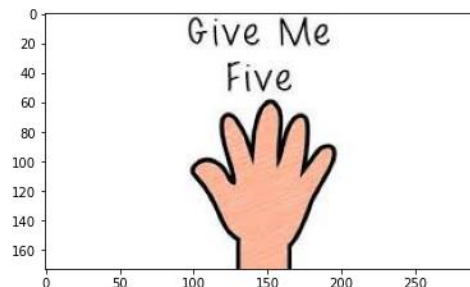


Parameters have been trained!
Train Accuracy: 0.9388889
Test Accuracy: 0.8416664

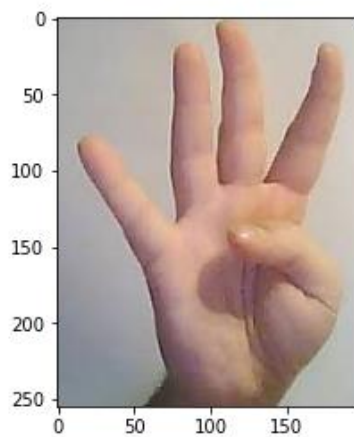
(四)結論

從以上幾個結果來看，在不改變神經架構的層數來看，Momentum 做為優化器應是最簡單暴力的訓練模型，在許多嘗試中(未改變學習次數)雖然無法達到正確率 90%，但是能輕易突破 80%。RMSPropOptimizer 和 AdagradOptimizer 相比之下學習緩慢，但只要將 learning rate 稍微加大，並增加學習次數，就能有很好的模型，遺憾的是，不管用甚麼模型測試以下圖片都無法正確辨認，期望以後使用 keras 能有正確率超過 95%的模型。

Your algorithm predicts: $y = 1$

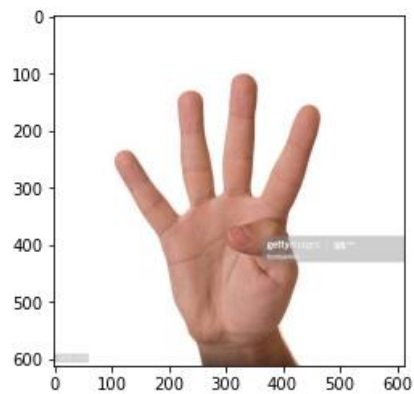
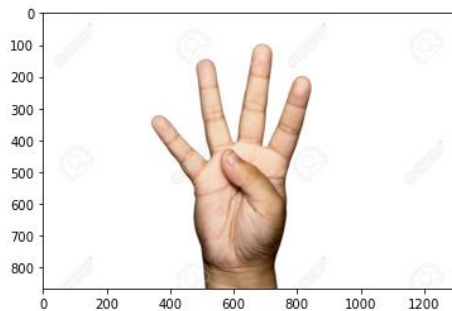


Your algorithm predicts: $y = 2$



Your algorithm predicts: $y = 0$

Your algorithm predicts: $y = 1$



另外做了小測試，將原本下載的圖片“YA”稍做加工，產出一個六根手指但是比“三”的獵奇圖片：



將此圖片丟進準確率較高的模型(正確率>80%)顯示出來是“2”，模型：
MomentumOptimizer(learning_rate=0.0001,momentum=0.5)，準確率：83%。

Your algorithm predicts: $y = 2$



將此獵奇圖片丟進準確率普普的模型($80\% > \text{正確率} > 70\%$)顯示出來是“4”。參數：
`AdagradOptimizer(learning_rate=0.0004, initial_accumulator_value=0.1)`，準確率
71%。

Your algorithm predicts: $y = 4$

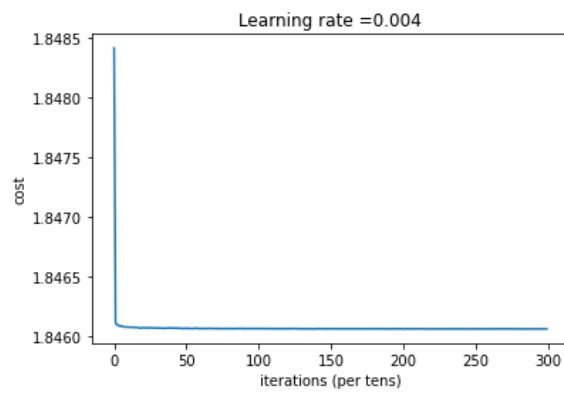


為了知道是否模型本身就不適合預測(因為正確率過低)，將正常的“YA”圖片做測試，是錯誤的預測，可知本身就不能使用。

Your algorithm predicts: $y = 4$



有趣的是，在修改參數與優化器的嘗試中，誕生了一些爛掉的模型，例如以下
`AdagradOptimizer(learning_rate=0.004, initial_accumulator_value=0.1)`



Parameters have been trained!
Train Accuracy: 0.1666667
Test Accuracy: 0.1666667

Your algorithm predicts: $y = 3$



丟進此爛掉的模型卻能正確的預測為 3 !